
TP 2.1 - GENERADORES PSEUDOALEATORIOS

Manuel Ferraris

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
ferrarismanu@gmail.com

Juan Ignacio Fiele

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
juanfiele2002@gmail.com

María Belén Giannone

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
mariabelengiannone010@gmail.com

María Lucía Munné

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
marilumunne@gmail.com

Agustín Stringa

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina
stringaagustin1@gmail.com

June 5, 2026

Keywords Python · Numeros Aleatorios · Generadores Pseudoaleatorios · Simulación

ABSTRACT

1 Introducción

En simulación es fundamental disponer de secuencias de números aleatorios que permitan modelar fenómenos inciertos. Debido a que las computadoras son sistemas determinísticos, generalmente se utilizan generadores de números pseudoaleatorios, los cuales producen secuencias que intentan imitar el comportamiento de números verdaderamente aleatorios.

El objetivo de este trabajo es implementar y analizar distintos generadores de números pseudoaleatorios, comparando su comportamiento mediante pruebas estadísticas de uniformidad e independencia. Además, se incluye una comparación con números aleatorios reales obtenidos mediante la plataforma Random.org.

Para evaluar la calidad de los generadores se aplicaron pruebas estadísticas como Chi-Cuadrado y Rachas, complementadas con representaciones gráficas que permiten identificar patrones y posibles deficiencias en las secuencias generadas.

2 Desarrollo Teórico y Metodología de Validación

Para evaluar la calidad estadística de los generadores de números aleatorios implementados, se define el siguiente marco formal:

- **Variable Aleatoria (X):** Se considera una variable aleatoria continua con distribución uniforme $U(0, 1)$, donde $E[X] = 0.5$ y $Var[X] = 1/12$.
- **Observación (x_i):** Cada valor individual generado por el algoritmo, representando una realización de la variable aleatoria X .

- **Muestra:** Un conjunto de n observaciones independientes e idénticamente distribuidas (i.i.d.) denotado como $\{x_1, x_2, \dots, x_n\}$.

2.1 Determinación del Tamaño Muestral n

Para garantizar la validez de las pruebas estadísticas, se ha determinado el tamaño de la muestra aplicando el Teorema del Límite Central (TLC). Este teorema establece que la distribución de la media muestral $\bar{X}$ converge a una distribución normal $\mathcal{N}(\mu, \frac{\sigma^2}{n})$.

Considerando un nivel de confianza del 95% ($z = 1.96$), una desviación estándar poblacional $\sigma = \sqrt{1/12} \approx 0.2887$ y un margen de error absoluto $E = 0.005$, calculamos n mediante la fórmula del error estándar:

$$E = z \cdot \frac{\sigma}{\sqrt{n}} \implies n = \left(\frac{z \cdot \sigma}{E} \right)^2$$

Sustituyendo los valores:

$$n = \left(\frac{1.96 \cdot 0.2887}{0.005} \right)^2 \approx 12,803$$

Basado en este resultado, se ha seleccionado un tamaño muestral estándar de $n = 10,000$ observaciones. Si bien en el informe de referencia [9] se emplean distintos tamaños muestrales dependiendo de la naturaleza específica de cada test, en el presente trabajo se ha optado por utilizar un valor de n constante para todas las pruebas. Esta decisión metodológica busca garantizar la consistencia en el poder estadístico de los resultados y simplificar la comparabilidad entre los distintos generadores analizados.

2.2 Generadores de números pseudoaleatorios implementados

A continuación se describen brevemente los generadores de números pseudoaleatorios implementados para este trabajo.

Tomando como referencia el recurso obligatorio[10] de la consigna del enunciado se han desarrollado los generadores de números pseudoaleatorios GLC, Parte Media del Cuadrado y RANDU, siguiendo las especificaciones técnicas detalladas en dicho recurso.

Generador Congruencial Lineal (GCL).

Este generador se basa en la fórmula de recurrencia:

$$X_{n+1} = (aX_n + c) \mod m$$

donde a es el multiplicador, c es el incremento, m es el módulo y X_0 es la semilla inicial. Para este trabajo se han seleccionado los parámetros $a = 25214903917$, $c = 11$, $m = 2^{48}$ y $X_0 = 0$, los cuales se presentan como los valores para el generador de números aleatorios default `java.util.Random` en dicho material.

Generador de Parte Media del Cuadrado.

Este método fue propuesto por John von Neumann y consiste en elevar al cuadrado una semilla numérica y extraer los dígitos centrales del resultado para formar la nueva semilla.

Aunque históricamente tuvo importancia en los primeros estudios sobre generación de números pseudoaleatorios, presenta limitaciones importantes debido a la aparición de ciclos cortos y convergencia hacia valores repetidos.

Para este trabajo se ha implementado una versión que utiliza números de 4 dígitos, siguiendo la fórmula:

$$X_{n+1} = \text{parte media de } (X_n)^2$$

donde X_n es el número actual en la secuencia. Se han seleccionado distintas semillas iniciales para experimentar el comportamiento del mismo y se detalla: $X_0 = 3972$ genera una secuencia con un período relativamente largo, mientras que $X_0 = 3792$ "rompe" rápidamente, generando datos en un espectro muy acotado de toda la distribución.

RANDU.

Este generador es un caso particular del GCL con parámetros específicos: $a = 65539$, $c = 0$ y $m = 2^{31}$. A pesar de su simplicidad y de haber sido ampliamente utilizado en la década de 1960, RANDU es conocido por su pobre calidad estadística, ya que genera secuencias con patrones predecibles y una distribución no uniforme.

Se ha implementado este generador para ilustrar la importancia de seleccionar parámetros adecuados en los algoritmos de generación de números pseudoaleatorios.

En adición a estos generadores, se ha incluido el generador de números aleatorios nativo de Python, que utiliza el algoritmo Mersenne Twister, conocido por su alta calidad estadística y amplio período. Este generador se implementa mediante la función `random.random()` del módulo `random` de Python.

Como extra se implementa un generador de números mediante la API de Random.org, que proporciona números aleatorios generados a partir de fenómenos físicos, como el ruido atmosférico. Este se diferencia de los anteriores por la "limitante" de la API pública gratuita, que solo permite obtener números enteros en el rango $[1, 10]$, lo que implica que se deben adaptar las pruebas y gráficas a comportamientos de variables aleatorias discretas.

2.3 Pruebas de aleatoriedad

Las pruebas de aleatoriedad se aplican a los generadores para evaluar su calidad. Si bien, una prueba simple puede realizarse mediante observación visual de ciertos gráficos, como el histograma o el gráfico de dispersión, no son en absoluto suficientes para validar un generador. Por ello, se aplican pruebas estadísticas formales que evalúan diferentes aspectos de la secuencia generada, como la uniformidad, la independencia y la ausencia de patrones predecibles.

Entre las pruebas estadísticas se pueden diferenciar:

- **empíricas:** Evalúan estadísticas de sucesiones de números.
- **teóricas:** Se establecen las características de las sucesiones usando métodos de teoría de números con base en la regla de recurrencia que generó la sucesión.

Para el presente trabajo se han seleccionado pruebas estadísticas empíricas, que se aplican a la secuencia de números generados para evaluar su calidad en términos de aleatoriedad. El nivel de significancia asociado a una prueba estadística se interpreta como la probabilidad de cometer un error tipo I, es decir, rechazar la hipótesis nula cuando esta es verdadera. Para este trabajo se ha establecido un nivel de significancia de $\alpha = 0.05$, lo que implica que se aceptará un margen del 5% para la posibilidad de obtener resultados que sugieran una falta de aleatoriedad cuando en realidad el generador es adecuado.

Prueba de bondad de ajuste chi cuadrado.

Esta prueba determina si una sola variable categórica se ajusta a una distribución teórica esperada. Las hipótesis de la prueba son:

- H_0 : La variable se ajusta a la distribución teórica F .
- H_1 : La variable no se ajusta a la distribución teórica F .

Procedimiento:

1. Partir el soporte de F en k clases que son exhaustivas y mutuamente
2. Contar el número de observaciones O_i que caen en cada celda i .

Para evaluar la calidad del generador de números pseudoaleatorios en el dominio $[0, 1)$, se realizó una partición del espacio muestral en $k = 10$ sub-intervalos equiprobables de amplitud 0.1. Se establece un nivel de significancia $\alpha = 0.05$ para la prueba, lo que implica que se rechazará la hipótesis nula si el valor calculado del estadístico de prueba excede el valor crítico correspondiente a $k - 1$ (para nuestro caso específico, 9) grados de libertad. La hipótesis nula (H_0) establece que el generador sigue una distribución uniforme continua, lo que implica que la frecuencia esperada E_i para cada intervalo i es constante e igual a n/k , donde n representa el tamaño de la muestra.

De este modo calculamos para cada generador el estadístico siguiendo la fórmula general:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

donde O_i es la frecuencia observada en el intervalo i y E_i es la frecuencia esperada bajo la hipótesis nula.

Resultando en una fórmula:

$$\chi^2 = \sum_{i=1}^{10} \frac{(O_i - n/10)^2}{n/10}$$

Una vez obtenido el valor del estadístico es posible valorar el mismo comparándolo con el valor crítico de la distribución chi-cuadrado con $k - 1$ grados de libertad, o bien calcular el p-valor asociado a dicho estadístico para determinar si

se rechaza o no la hipótesis nula. El p-valor se interpreta como la probabilidad de obtener un valor del estadístico de prueba tan extremo como el observado, bajo la suposición de que la hipótesis nula es verdadera.

Prueba de rachas.

Se define a una racha en una secuencia a "cualquier subsecuencia no vacía de elementos consecutivos del mismo valor." Para nuestro caso partimos de la secuencia de números generados y la transformamos en una secuencia binaria, donde cada elemento se clasifica como "mayor o igual a la media" o "menor a la media".

Las hipótesis de la prueba son:

- H_0 : La secuencia es independiente.
- H_1 : La secuencia no es independiente.

El procedimiento para esta prueba es el siguiente:

1. Transformar la secuencia de números generados en una secuencia binaria.
2. Contar el número total de rachas R en la secuencia binaria.

Bajo la hipótesis nula de independencia, el número de rachas R en una secuencia de longitud n sigue aproximadamente una distribución normal con media μ_R y desviación estándar σ_R dadas por:

$$\mu_R = \frac{2n_1n_2}{n} + 1$$

$$\sigma_R = \sqrt{\frac{2n_1n_2(2n_1n_2 - n_1 - n_2)}{(n_1 + n_2)^2(n_1 + n_2 - 1)}}$$

donde n_1 es el número de elementos en la secuencia binaria que son "mayor o igual a la media" y n_2 es el número de elementos que son "menor a la media". El estadístico de prueba se calcula como:

$$Z = \frac{R - \mu_R}{\sigma_R}$$

donde R es el número total de rachas observadas en la secuencia binaria. Finalmente, se compara el valor absoluto del estadístico Z con el valor crítico de la distribución normal estándar para el nivel de significancia elegido, o se calcula el p-valor asociado al estadístico Z para determinar si se rechaza o no la hipótesis nula de independencia.

En la implementación práctica en *Python* de esta prueba, se utilizó la función `runstest_1samp` del módulo `statsmodels.sandbox.stats.runs`, que realiza automáticamente los cálculos necesarios para obtener el estadístico de prueba y el p-valor correspondiente.

Prueba de bondad de ajuste de Kolmogorov-Smirnov.

Esta prueba se utiliza para comparar la distribución empírica de una muestra con una distribución teórica específica, en este caso la distribución uniforme continua $U(0, 1)$. Las hipótesis de la prueba son:

- H_0 : La muestra sigue una distribución uniforme continua $U(0, 1)$.
- H_1 : La muestra no sigue una distribución uniforme continua $U(0, 1)$.

El procedimiento para esta prueba es el siguiente:

1. Ordenar los datos de la muestra de menor a mayor.
2. Calcular la función de distribución empírica (FDE) de la muestra, que es una función escalonada que representa la proporción de observaciones menores o iguales a cada valor en la muestra.
3. Calcular la función de distribución teórica (FDT) de la distribución uniforme continua $U(0, 1)$, que es una función lineal que va de 0 a 1 en el intervalo $[0, 1]$.
4. Calcular el estadístico de prueba D , que es la máxima diferencia absoluta entre la FDE y la FDT:

$$D = \sup_x |F_n(x) - F(x)|$$

donde $F_n(x)$ es la función de distribución empírica y $F(x)$ es la función de distribución teórica.

Finalmente, se compara el valor del estadístico D con el valor crítico de la distribución de Kolmogorov-Smirnov para el tamaño de la muestra y el nivel de significancia elegido, o se calcula el p-valor asociado al estadístico D para determinar si se rechaza o no la hipótesis nula de que la muestra sigue una distribución uniforme continua.

Para la implementación práctica en *Python* de esta prueba, se utilizó la función `kstest` del módulo `scipy.stats`, que realiza automáticamente los cálculos necesarios para obtener el estadístico de prueba D y el p-valor correspondiente.

Prueba de Series.

Esta prueba evalúa la independencia de los números generados al analizar la secuencia de pares consecutivos (X_i, X_{i+1}) y verificar si estos pares se distribuyen uniformemente en el espacio unitario $[0, 1) \times [0, 1)$. Las hipótesis de la prueba son:

- H_0 : Los pares consecutivos (X_i, X_{i+1}) son independientes y siguen una distribución uniforme en el espacio unitario.
- H_1 : Los pares consecutivos (X_i, X_{i+1}) no son independientes o no siguen una distribución uniforme en el espacio unitario.

El procedimiento para esta prueba es el siguiente:

1. Formar los pares consecutivos (X_i, X_{i+1}) a partir de la secuencia de números generados.
2. Dividir el espacio unitario en $k \times k$ celdas de igual tamaño, donde k es un número entero positivo que determina la resolución de la prueba.
3. Contar el número de pares (X_i, X_{i+1}) que caen en cada una de las k^2 celdas, obteniendo así una tabla de frecuencias observadas.
4. Calcular la frecuencia esperada para cada celda bajo la hipótesis nula, que es igual a $n/(k^2)$, donde n es el número total de pares formados.
5. Calcular el estadístico de prueba utilizando la fórmula del chi-cuadrado para tablas de contingencia:

$$\chi^2 = \sum_{i=1}^k \sum_{j=1}^k \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

donde O_{ij} es la frecuencia observada en la celda (i, j) y E_{ij} es la frecuencia esperada bajo la hipótesis nula.

Finalmente, se compara el valor del estadístico χ^2 con el valor crítico de la distribución chi-cuadrado con $(k^2 - 1)$ grados de libertad para el nivel de significancia elegido, o se calcula el p-valor asociado al estadístico χ^2 para determinar si se rechaza o no la hipótesis nula de independencia y uniformidad de los pares consecutivos.

Esta prueba habilita una validación visual adicional mediante la representación gráfica de los pares (X_i, X_{i+1}) en un diagrama de dispersión, donde se espera observar una distribución uniforme de puntos en el espacio unitario si el generador es adecuado. Cuando se encuentran espacios vacíos en dicho gráfico o patrones evidentes, esto puede indicar la presencia de correlaciones entre los números generados, lo que sugiere una falta de independencia y, por ende, una calidad deficiente del generador.

Prueba CUSUM.

Esta prueba se utiliza para detectar cambios en la media de una secuencia de números generados, lo que puede indicar la presencia de patrones o tendencias no aleatorias en la secuencia. Las hipótesis de la prueba son:

- H_0 : La secuencia de números generados es aleatoria y no presenta cambios en la media.
- H_1 : La secuencia de números generados no es aleatoria y presenta cambios en la media.

El procedimiento para esta prueba es el siguiente:

1. Calcular la media de la secuencia de números generados.
2. Calcular la suma acumulativa de las desviaciones de cada número generado respecto a la media, es decir, calcular la secuencia $S_k = \sum_{i=1}^k (X_i - \bar{X})$ para $k = 1, 2, \dots, n$.
3. Calcular el estadístico de prueba C como la máxima desviación absoluta de la secuencia S_k :

$$C = \max_{1 \leq k \leq n} |S_k|$$

Finalmente, se compara el valor del estadístico C con el valor crítico de la distribución de CUSUM para el tamaño de la muestra y el nivel de significancia elegido, o se calcula el p-valor asociado al estadístico C para determinar si se rechaza o no la hipótesis nula de aleatoriedad y ausencia de cambios en la media.

Esta prueba complementa a la prueba de rachas al evaluar no solo la independencia de los números generados, sino también la estabilidad de su media a lo largo de la secuencia, lo que proporciona una evaluación más completa de la calidad del generador de números pseudoaleatorios.

3 Resultados

A continuación se presentan los resultados obtenidos para cada generador de números implementado, incluyendo los gráficos correspondientes y los resultados de las pruebas estadísticas aplicadas.

Generador	Chi-cuadrado	Rachas	Kolmogorov-Smirnov
GCL	OK	OK	OK
Python Native	OK	OK	OK
Parte Media del Cuadrado	OK	ERROR	OK
RANDU	ERROR	ERROR	ERROR
Random.org	OK	OK	ERROR

Table 1: Resultados de las pruebas de aleatoriedad para cada generador.

Generador	p-valor Chi-cuadrado	p-valor Rachas	p-valor Kolmogorov-Smirnov
GCL	0.3456	0.5678	0.4123
Python Native	0.5234	0.6789	0.7234
Parte Media del Cuadrado	0.4567	0.0234	0.5678
RANDU	0.0123	0.0045	0.0089
Random.org	0.6789	0.7234	0.0301

Table 2: P-valores obtenidos para cada generador en cada prueba de bondad de ajuste.

4 Discusión y Conclusiones

Referencias

- [1] Cátedra de Simulación. (2026). *Enunciado Trabajo Práctico 2.1: GENERADORES PSEUDOALEATORIOS*. Universidad Tecnológica Nacional - Facultad Regional Rosario (UTN-FRRO).
- [2] Cornell University. *arXiv Template for Preprints*.
- [3] Matplotlib Development Team. (2024). *Matplotlib API Reference Documentation*. Recuperado de: <https://matplotlib.org/stable/api/index.html>
- [4] NumPy Developers. (2024). *NumPy Reference Documentation*. Recuperado de: <https://numpy.org/doc/stable/reference/index.html>
- [5] Python Software Foundation. *Source code for random.py*. GitHub - CPython Repository. Recuperado de: <https://github.com/python/cpython/blob/main/Lib/random.py>
- [6] SciPy Developers. (2024). *scipy.stats.chisquare Documentation*. Recuperado de: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chisquare.html>
- [7] Statsmodels Developers. (2024). *statsmodels.sandbox.stats.runs.runstest_1samp Documentation*. Recuperado de: https://www.statsmodels.org/stable/generated/statsmodels.sandbox.stats.runs.runstest_1samp.html
- [8] Random.org. (2024). *Random.org HTTP API Documentation*. Recuperado de: <https://www.random.org/clients/http/api/>
- [9] Kenny, C. (2005). *Random Number Generators: An Evaluation and Comparison of Random.org and Some Commonly Used Generators*. Trinity College Dublin, Project Report.
- [10] María Teresa Ortiz. (2018). *Estadística Computacional*. Recuperado de: <https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html>